

1 Proof Techniques

Efficient Algorithms · Wintersemester 2026–27

Prof. Dr. Sebastian Wild

CS566 (Wintersemester 2026–27)
Philipps-Universität Marburg
version 2026-10-13 14:01

Learning Outcomes

Unit 1: ***Proof Techniques***

1. Know logical *proof strategies* for proving implications, set inclusions, set equalities, and quantified statements.
2. Be able to use *mathematical induction* in simple proofs.
3. Know techniques for *proving termination* and *correctness* of procedures.

Outline

1 Proof Techniques

1.1 Digression: Random Shuffle

1.2 Proof Templates

1.3 Mathematical Induction

1.4 Correctness Proofs

1.1 Digression: Random Shuffle

Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.

- A natural approach yields the following code

```
1 procedure myShuffle(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4         Swap A[i] and A[r] // Swap A[i] to random position.
5     end for
```

- Intuitively: *All elements are moved to a random index, so the order is random . . . right?*

Clicker Question



Select all statements that apply to myShuffle (for you).

- **A** I have seen this shuffling algorithm (or a very similar method) before.
- **B** I can understand the pseudocode for myShuffle (so I would be able to do an example by hand).
- **C** It can generate all possible orderings of A (depending on the random numbers).
- **D** myShuffle produces all possible orderings with the same probability.
- **E** Assuming randomInt gives (perfect) uniform random numbers in the given range, myShuffle generates any ordering with equal probability.



→ *sli.do/effalg*

Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.
- A natural approach yields the following code

```
1 procedure myShuffle(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4         Swap A[i] and A[r] // Swap A[i] to random position.
5     end for
```

- Intuitively: *All elements are moved to a random index, so the order is random . . . right?*



$n = 2$

Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.
- A natural approach yields the following code

```
1  procedure myShuffle(A[0..n))
2      for i := 0, ..., n − 1
3          r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4          Swap A[i] and A[r] // Swap A[i] to random position.
5      end for
```

- Intuitively: *All elements are moved to a random index, so the order is random . . . right??*



$n = 2$ $n = 3$

Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.
- A natural approach yields the following code

```
1 procedure myShuffle(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4         Swap A[i] and A[r] // Swap A[i] to random position.
5     end for
```

- Intuitively: *All elements are moved to a random index, so the order is random . . . right???*



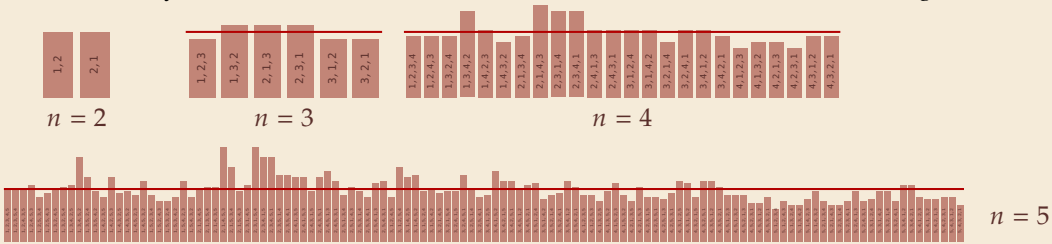
$n = 2$ $n = 3$ $n = 4$

Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.
- A natural approach yields the following code

```
1 procedure myShuffle(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4         Swap A[i] and A[r] // Swap A[i] to random position.
5     end for
```

- Intuitively: *All elements are moved to a random index, so the order is random . . . right????*



Random shuffle

- **Goal:** Put an array $A[0..n)$ of n numbers into random order.
  More precisely: Any ordering of the elements $A[0], \ldots, A[n-1]$ should be equally likely.
- A natural approach yields the following code

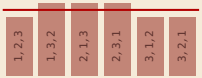
```
1 procedure myShuffle(A[0..n))
2     for i := 0, ..., n − 1
3         r := randomInt([0..n)) // A uniformly random number r with 0 ≤ r < n.
4         Swap A[i] and A[r] // Swap A[i] to random position.
5     end for
```

← **WRONG! DO NOT USE**

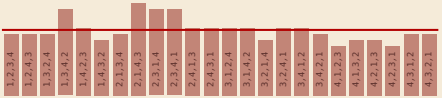
- Intuitively: *All elements are moved to a random index, so the order is random . . . right????*



$n = 2$



$n = 3$



$n = 4$



$n = 5$

Clicker Question

Select all statements that apply to myShuffle (for you).



A I have seen this shuffling algorithm (or a very similar method) before. ✓

B I can understand the pseudocode for myShuffle (so I would be to do an example by hand). ✓

C It can generate all possible orderings of A (depending on the random numbers). ✓

D ~~myShuffle produces all possible orderings with the same probability.~~

E ~~Assuming randomInt gives (perfect) uniform random numbers in the given range, myShuffle generates any ordering with equal probability.~~

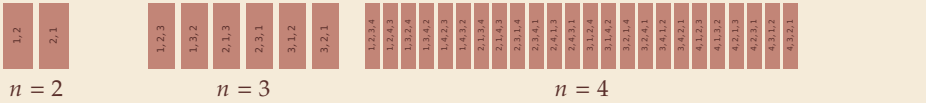


→ *sli.do/effalg*

Correct shuffle

- interestingly, a very small change corrects the issue

```
1  procedure shuffleKnuthFisherYates(A[0..n))
2      for i := 0, ..., n − 1
3          r := randomInt([i..n))
4          Swap A[i] and A[r]
5      end for
```



- looks good . . .
- . . . but how can we convince ourselves that it is correct, *beyond any doubt?*

1.2 Proof Templates

What is a *formal* proof?

A formal proof (in a logical system) is a **sequence of statements** such that each statement

1. is an *axiom* (of the logical system), or
2. follows from previous statements using the *inference rules* (of the logical system).

Among experts: Suffices to *convince a human* that a formal proof *exists*.

But: Use formal logic as guidance against faulty reasoning. $\rightsquigarrow$ bulletproof



What is a *formal* proof?

A formal proof (in a logical system) is a **sequence of statements** such that each statement

1. is an *axiom* (of the logical system), or
2. follows from previous statements using the *inference rules* (of the logical system).

Among experts: Suffices to *convince a human* that a formal proof *exists*.

But: Use formal logic as guidance against faulty reasoning. $\rightsquigarrow$ bulletproof



Notation:

- Statements: $A \equiv$ "it rains", $B \equiv$ "the street is wet".
- Negation: $\neg A$ "Not A"
- And/Or: $A \wedge B$ "A and B"; $A \vee B$ "A or B or both" (non-exclusive or)
- Implication: $A \Rightarrow B$ "If A, then B"; $\neg A \vee B$
- Equivalence: $A \Leftrightarrow B$ "A holds true *if and only if ('iff')* B holds true."; $(A \Rightarrow B) \wedge (B \Rightarrow A)$

Clicker Question



Is the following statement true?

"If the Earth is flat, then ships can fall over its rim."

$\neg A \vee B$

A

B

A Yes **B** No **C** Neither



→ *sli.do/effalg*

Clicker Question



Is the following statement true?
"If the Earth is flat, then ships can fall over its rim."

A Yes ✓ **B** ~~No~~ **C** ~~Neither~~



→ *sli.do/effalg*

Implications

$A \Rightarrow B \quad \equiv \boxed{\neg A \vee B}$

To prove $A \Rightarrow B$, we can

$\neg(\neg B) \vee (\neg A) \equiv \neg A \vee B$

- directly derive B from A *direct proof*
- prove $(\neg B) \Rightarrow (\neg A)$ *proof by contraposition* — Kontraposition
- assume $A \wedge \neg B$ and derive a contradiction *proof by contradiction, reductio ad absurdum* — Widerspruchsbeweis
- distinguish cases, i. e., separately prove $(A \wedge C) \Rightarrow B$ and $(A \wedge \neg C) \Rightarrow B$. *proof by exhaustive case distinction*

Clicker Question

$n = 2 \cdot k + 1$

$k \in \mathbb{N}_0$

$n^2 = (2k+1)^2$

$= 4k^2 + 4k + 1$

$= 2k' + 1$

$k' = 2k^2 + 2k$

$\in \mathbb{N}_0$

Suppose we want to prove:
"If $n^2 \in \mathbb{N}_0$ is an even number, then n is also even."
For that we show that when n is odd, also n^2 is odd.
Which proof template do we follow?



- **A** direct proof: $A \Rightarrow B$
- **B** proof by contraposition: $(\neg B) \Rightarrow (\neg A)$
- **C** proof by contradiction: $A \wedge \neg B \Rightarrow \lightning$
- **D** proof by case distinction: $(A \wedge C) \Rightarrow B$ and $(A \wedge \neg C) \Rightarrow B$



→ sli.do/effalg

Clicker Question



Suppose we want to prove:
"If $n^2 \in \mathbb{N}_0$ is an even number, then n is also even."
For that we show that when n is odd, also n^2 is odd.
Which proof template do we follow?

A ~~direct proof: $A \Rightarrow B$~~

B proof by contraposition: $(\neg B) \Rightarrow (\neg A)$ ✓

C ~~proof by contradiction: $A \wedge \neg B \Rightarrow \lightning$~~

D ~~proof by case distinction: $(A \wedge C) \Rightarrow B$ and $(A \wedge \neg C) \Rightarrow B$~~



→ sli.do/effalg

Equivalences

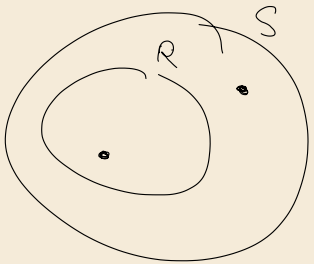
To prove $A \Leftrightarrow B$,
we prove both implications $A \Rightarrow B$ and $B \Rightarrow A$ separately.

(Often, one direction is much easier than the other.)

Set Inclusion and Equality

To prove that a set S contains a set R, i. e., $R \subseteq S$,
we prove the implication $x \in R \Rightarrow x \in S$.

To prove that two sets S and R are equal, $S = R$,
we prove both inclusions, $S \subseteq R$ and $R \subseteq S$ separately.



1.3 Mathematical Induction

Quantified Statements

$A(10)$ $\quad \neg A(5)$

Notation

- Statements with parameters: $A(x)$ ≡ "x is an even number."
- Existential quantifiers: $\exists x : A(x)$ "There exists some x, so that $A(x)$."
- Universal quantifiers: $\forall x : A(x)$ "For all x it holds that $A(x)$."

  Note: $\forall x : A(x)$ is equivalent to $\neg\exists x : \neg A(x)$

$\forall n \in \mathbb{N}_0 \; : \; \neg A(2n+1)$

Quantifiers can be nested, e. g., *ε-δ-criterion for limits*:

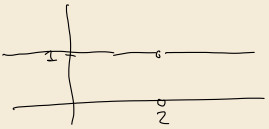
$$\lim_{x\to\xi} f(x) = a \quad :\Leftrightarrow \quad \forall \varepsilon > 0 \; \exists \delta > 0 \; \forall x \; : \; \big(0 < |x-\xi| < \delta\big) \Rightarrow \big|f(x) - a\big| < \varepsilon.$$

$\lim_{x\to 2} f(x) = 1$

(Order matters: δ may depend on ε, but not on x.)

$f(x) = \frac{x-2}{x-2}$ $\quad f(2)$ undefined!

To prove $\exists x : A(x)$, we simply list an example ξ such that $A(\xi)$ is true.



Clicker Question



Have you seen **proofs by *mathematical induction*** before?

- **A** Yes, could do it
- **B** Yes, but only vaguely remember
- **C** I've heard this term before, but ...
- **D** I have not heard "mathematical induction" before

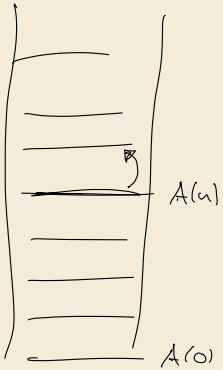


→ *sli.do/effalg*

For-all statements

To prove $\forall x : A(x)$, we can

- derive $A(x)$ for an *"arbitrary but fixed value of x"*, or,
- for $x \in \mathbb{N}_0$, use ***induction***, i. e.,
  - prove $A(0)$, *induction basis*, and
  - prove $\forall n \in \mathbb{N}_0 : A(n) \Rightarrow A(n+1)$ *inductive step*



For-all statements

To prove $\forall x : A(x)$, we can

- derive $A(x)$ for an *"arbitrary but fixed value of x"*, or,
- for $x \in \mathbb{N}_0$, use ***induction***, i. e.,
  - prove $A(0)$, *induction basis*, and
  - prove $\forall n \in \mathbb{N}_0 : A(n) \Rightarrow A(n+1)$ *inductive step*

More general variants of induction:

- complete/strong induction
  inductive step shows $\big(A(0) \wedge \cdots \wedge A(n)\big) \Rightarrow A(n+1)$
- structural/transfinite induction Noethersche Ordnung / wohlgeordnet
  works on any *well-founded* set, e. g., binary trees, graphs, Boolean formulas, strings, ...

„wohlgeordnet", no infinite strictly decreasing chains

1.4 Correctness Proofs

Formal verification

- verification: prove that a program computes the correct result

⇝ not our key focus in CS 566
but same techniques are useful for *reasoning* about algorithms

Here:

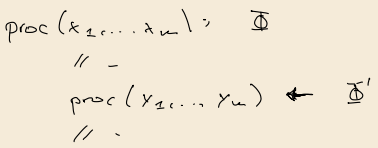
1. Prove that loop or recursive call eventually *terminates*.
2. Prove that a *loop* computes the *correct* result.

Proving termination

To prove that a recursive procedure $\texttt{proc}(x_1, \ldots, x_m)$ eventually terminates, we

- define a *potential* $\Phi(x_1, \ldots x_m) \in \mathbb{N}_0$ of the parameters
  (Note: $\Phi(x_1, \ldots x_m) \geq 0$ by definition!)
- prove that every recursive call decreases the potential, i. e.,
  any recursive call $\texttt{proc}(y_1, \ldots, y_m)$ inside $\texttt{proc}(x_1, \ldots, x_m)$ satisfies

$$\Phi(y_1, \ldots, y_m) < \Phi(x_1, \ldots, x_m) \qquad \text{which means}$$
$$\Phi(y_1, \ldots, y_m) \leq \Phi(x_1, \ldots, x_m) - 1$$



Proving termination

To prove that a recursive procedure $\texttt{proc}(x_1, \ldots, x_m)$ eventually terminates, we

- define a *potential* $\Phi(x_1, \ldots x_m) \in \mathbb{N}_0$ of the parameters
  (Note: $\Phi(x_1, \ldots x_m) \geq 0$ by definition!)
- prove that every recursive call decreases the potential, i. e.,
  any recursive call $\texttt{proc}(y_1, \ldots, y_m)$ inside $\texttt{proc}(x_1, \ldots, x_m)$ satisfies

$$\Phi(y_1, \ldots, y_m) < \Phi(x_1, \ldots, x_m) \qquad \text{which means}$$
$$\Phi(y_1, \ldots, y_m) \leq \Phi(x_1, \ldots, x_m) - 1$$

$\rightsquigarrow$ $\texttt{proc}(x_1, \ldots, x_m)$ terminates because
we can only strictly *decrease* the (integral) potential
a *finite* number of times from its initial value

- Can use same idea for a loop: show that potential decreases in each iteration.
  - $\rightsquigarrow$ see tutorials for an example.

Loop invariants

Goal: Prove that a *post condition* holds after execution of a (terminating) loop.

```
1 // (A) before loop
2 while cond do
3     // (B) before body
4     body
5     // (C) after body
6 end while
7 // (D) after loop
```

For that, we

- find a *loop invariant* I (that's the tough part!)
- prove that I holds at (A)
- prove that $I \wedge cond$ at (B) imply I at (C)
- prove that $I \wedge \neg cond$ imply the desired post condition at (D)

Note: I holds before, during, and after the loop execution, hence the name.

Hoare Calculus / Hoare Kalkül

{precondition}
P
{post condition}

≡

if precondition holds before P,
then after executing P the
post condition holds

Loop invariant – Example

- loop condition: $cond \equiv j < n$
- post condition (in line 13):
  $curMax = \max_{k \in [0..n-1]} A[k]$
- loop invariant:
  $I \equiv curMax = \max_{k \in [0..j-1]} A[k] \wedge j \leq n$

```
1  procedure arrayMax(A,n)
2      // input: array of n elements, n ≥ 1
3      // output: the maximum element in A[0..n − 1]
4      curMax := A[0]; j := 1
5      // (A)
6      while j < n do
7          // (B)
8          if A[j] > curMax
9              curMax := A[j]
10         j := j + 1
11         // (C)
12     end while
13     // (D)
14     return curMax
```

We have to prove:

(i) I holds at (A)

(ii) $I \wedge cond$ at (B) $\Rightarrow$ I at (C)

(iii) $I \wedge \neg cond \Rightarrow$ post condition

```
1  // (A) before loop
2  while cond do
3      // (B) before body
4      body
5      // (C) after body
6  end while
7  // (D) after loop
```

For that, we

- find a *loop invariant I* (that's the tough part!)
- prove that I holds at (A)
- prove that $I \wedge cond$ at (B) imply I at (C)
- prove that $I \wedge \neg cond$ imply the desired post condition at (D)

Loop invariant – Example

```
1  procedure arrayMax(A,n)
2      // input: array of n elements, n ≥ 1
3      // output: the maximum element in A[0..n − 1]
4      curMax := A[0]; j := 1
5      // (A)
6      while j < n do
7          // (B)
8          if A[j] > curMax
9              curMax := A[j]
10         j := j + 1
11         // (C)
12     end while
13     // (D)
14     return curMax
```

(ii) $I \wedge cond$ at (B) $\Rightarrow$ I at (C)

(iii) $I \wedge \neg cond$ $\Rightarrow$ post condition

(i) $curMax = \max_{k \in [0..j)} A[k] \wedge j \leq n \wedge j < n$

Case 1: $A[j] \leq curMax$

$A[j] \leq cu.Max = \max_{k \in [0..j)} A[k] \overset{(I)}{=} \max_{k \in [0..j+1)} A[k]$ // line 9

after line 10 $curMax = \max_{k \in [0..j)} A[k]$

$j < n$ at (B) $\Rightarrow$ $j \leq n-1$ // line 9

after line 10 $j \leq n$

$\Rightarrow$ I restored at (C)

Case 2 : $A[j] > \text{cur Max}$

$$A[j] = \max_{k \in [0..j+1)} A[k]$$

after line 9 : $\text{cur Max} = A[j]$

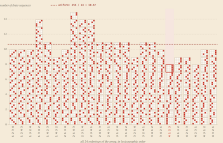
rest identical to Case 1 I restored at (C)

Shuffling: beyond any doubt

- **myShuffle cannot be uniform** (for $n \geq 3$):
  - the n calls of `randomInt([0..n))` yield n^n equally likely sequences $(r_0, \ldots, r_{n-1})$
  - each sequence determines the output ordering

  $\rightsquigarrow \Pr[\text{ordering } \pi] = \dfrac{\#\text{sequences producing } \pi}{n^n}$ is a multiple of $\dfrac{1}{n^n}$
  - uniform would mean $\Pr[\pi] = \dfrac{1}{n!}$ for all π, i. e., $n! \mid n^n$
  - but $n-1$ divides $n!$ and $\gcd(n-1, n) = 1$, so $(n-1) \nmid n^n$ for $n \geq 3$ (contraposition!)

Shuffling: beyond any doubt



- **myShuffle cannot be uniform** (for $n \geq 3$):
  - the n calls of `randomInt(`$[0..n)$`)` yield n^n equally likely sequences $(r_0, \ldots, r_{n-1})$
  - each sequence determines the output ordering

  $\rightsquigarrow \Pr[\text{ordering } \pi] = \dfrac{\#\text{sequences producing } \pi}{n^n}$ is a multiple of $\dfrac{1}{n^n}$
  - uniform would mean $\Pr[\pi] = \dfrac{1}{n!}$ for all π, i. e., $n! \mid n^n$
  - but $n-1$ divides $n!$ and $\gcd(n-1, n) = 1$, so $(n-1) \nmid n^n$ for $n \geq 3$ (contraposition!)

- **shuffleKnuthFisherYates is uniform:**



  - now `randomInt(`$[i..n)$`)` has $n - i$ options, so there are $n \cdot (n-1) \cdots 1 = n!$ equally likely sequences
  - *Claim:* different sequences produce different orderings.

    Induction on n: iteration $i = 0$ puts $A[r_0]$ in position 0, so $A[0]$ reveals r_0;
    the remaining iterations are the same algorithm on $A[1..n)$, so by induction they reveal $r_1, \ldots, r_{n-1}$.

  $\rightsquigarrow$ $n!$ sequences, $n!$ orderings, injective $\rightsquigarrow$ bijection
  $\rightsquigarrow$ every ordering has probability exactly $\frac{1}{n!}$ ■